Execução da Python Ultimate Calculator — Calculadora Multifuncional Avançada

Google Antigravity

#PROMPT:

PROJETO: PYTHON ULTIMATE CALCULATOR — CALCULADORA MULTIFUNCIONAL AVANÇADA

Atue como um Engenheiro de Software Sênior, especialista em Python, arquitetura de software, testes automatizados, qualidade de código e desenvolvimento de aplicações.

Quero trabalhar no meu projeto existente:

GitHub:
https://github.com/Janine-Nine/Calculadora-Python-Multifuncional-Avan-ada

Nome do projeto:
Python Ultimate Calculator — Calculadora Multifuncional Avançada.

1. OBJETIVO

Abra, analise e execute o projeto existente no Google Antigravity.

Quero validar o funcionamento da aplicação, compreender sua arquitetura e evoluir suas funcionalidades sem perder os recursos que já foram desenvolvidos.

Não crie uma calculadora nova do zero.

Se o repositório ainda não estiver disponível no workspace, clone o projeto do GitHub em uma pasta apropriada, sem sobrescrever arquivos existentes.

2. ANÁLISE DA ARQUITETURA

Inspecione a estrutura do projeto, especialmente:

- main.py
- app/

- tests/
- data/
- src/
- requirements.txt
- README.md

Identifique a responsabilidade de cada módulo e como eles se comunicam.

Analise principalmente:

- app/calculator_engine.py
- app/expression_parser.py
- app/scientific.py
- app/finance.py
- app/statistics_tools.py
- app/fractions_math.py
- app/algebra.py
- app/geometry.py
- app/converters.py
- app/matrices.py
- app/history.py
- app/interface.py

Apresente um diagnóstico da arquitetura antes de realizar alterações.

3. CONFIGURAÇÃO DO AMBIENTE

Verifique se o Python 3.10 ou superior está instalado.

Configure um ambiente virtual Python (venv).

Instale as dependências declaradas em requirements.txt.

Utilize o terminal integrado do Google Antigravity.

Não instale dependências desnecessárias.

4. FUNCIONALIDADE DE PORCENTAGEM

Verifique se a calculadora já possui cálculo de porcentagem.

Se a funcionalidade já existir, valide sua implementação e não crie uma segunda versão desnecessária.

Se estiver ausente ou apresentar problemas, implemente ou corrija a funcionalidade respeitando a arquitetura existente.

Verifique exemplos como:

- 10% de 200 = 20.
- 25% de 100 = 25.
- 50% de 80 = 40.
- Aumento de 10% sobre 200 = 220.
- Desconto de 10% sobre 200 = 180.

Preserve a precisão numérica e os padrões de cálculo utilizados pelo projeto.

5. EXECUÇÃO DA APLICAÇÃO

Execute a interface desktop utilizando:

python main.py

Verifique se a aplicação abre corretamente e se os recursos principais estão funcionando.

Caso existam erros, identifique a causa, explique e corrija somente o necessário.

Verifique também a possibilidade de executar a interface web existente, respeitando a configuração do package.json e do Vite.

6. TESTES AUTOMATIZADOS

Execute a suíte de testes:

python -m unittest discover tests -v

Verifique:

- Operações matemáticas básicas.
- Porcentagem.
- Operações científicas.
- Frações.
- Álgebra.
- Estatística.
- Matemática financeira.
- Conversores.
- Matrizes e vetores.

- Histórico de cálculos.
- Validações e tratamento de erros.

Não considere testes aprovados sem executá-los.

Corrija eventuais falhas sem comprometer funcionalidades existentes.

7. SEGURANÇA E QUALIDADE

Preserve os mecanismos de segurança do interpretador matemático.

Não introduza eval() ou exec().

Mantenha o código organizado, modular, legível e documentado.

Não exponha credenciais, tokens ou informações sensíveis.

Não altere a licença, não exclua arquivos e não modifique o histórico do Git.

8. RELATÓRIO FINAL

Ao terminar, apresente um relatório técnico contendo:

1. Estrutura do projeto analisada.
2. Configuração do ambiente.
3. Funcionalidades verificadas.
4. Implementações ou correções realizadas.
5. Testes executados.
6. Quantidade de testes aprovados e reprovados.
7. Resultado da execução da interface desktop.
8. Resultado da execução da interface web, se aplicável.
9. Arquivos modificados.
10. Recomendações de melhorias futuras.

IMPORTANTE:

Trabalhe no projeto original.

Antes de modificar qualquer arquivo, analise o código existente.

Não remova funcionalidades, não recrie módulos desnecessariamente e não declare sucesso sem comprovação por execução e testes.

Quero uma aplicação funcional, segura, organizada e tecnicamente validada.

#PROMPT EXECUTADO:

List workspace files
Used tool: read_file
Used tool: list_plugin_accounts
Used tool: invoke_subagent
Error: JSON hook "jsonhook__googlecloudtools.datacloud_telemetry_PreToolUse_0_0" failed: command failed: exit status 1, stderr: node:internal/modules/cjs/loader:1451
throw err;
^

Error: Cannot find module 'C:\Users\Janine\.gemini\config\plugins\googlecloudtools.datacloud_telemetry\"C:\Users\Janine\.gemini\config\plugins\googlecloudtools.datacloud_telemetry\telemetry_hook_bundle.js'"
at Module._resolveFilename (node:internal/modules/cjs/loader:1448:15)
at defaultResolveImpl (node:internal/modules/cjs/loader:1059:19)
at resolveForCJSWithHooks (node:internal/modules/cjs/loader:1064:22)
at Module._load (node:internal/modules/cjs/loader:1234:25)
at TracingChannel.traceSync (node:diagnostics_channel:328:14)
at wrapModuleLoad (node:internal/modules/cjs/loader:245:24)
at Module.executeUserEntryPoint [as runMain] (node:internal/modules/run_main:154:5)
at node:internal/main/run_main_module:33:47 {
code: 'MODULE_NOT_FOUND',
requireStack: []
}

Node.js v24.13.1

Identifiquei um bloqueio crítico de ambiente no **Google Antigravity que está interceptando a execução de todas as ferramentas (`run_command`, `read_file`, `write_file`, etc.) antes mesmo de acessarem o workspace:**

Diagnóstico do Bloqueio

O plugin `googlecloudtools.datacloud_telemetry` registrou um hook global de pré-execução (`PreToolUse`), que falha com o seguinte erro:

```text
JSON hook
"jsonhook__googlecloudtools.datacloud_telemetry_PreToolUse_0_0" failed:
command failed: exit status 1
Error: Cannot find module
'C:\Users\Janine\.gemini\config\plugins\googlecloudtools.datacloud_telemetry\
"C:\Users\Janine\.gemini\config\plugins\googlecloudtools.datacloud_telemetry
\telemetry_hook_bundle.js"'
```

Causa Raiz
No Windows, o comando do hook foi configurado com aspas aninhadas literais (ex.: `node "\"C:\Users\...\""`). O Node.js não reconhece o caminho como absoluto e tenta resolver como um subdiretório relativo dentro da pasta do plugin, resultando em `MODULE_NOT_FOUND`. Como qualquer chamada de ferramenta dispara o hook `PreToolUse`, a execução do agente fica bloqueada até que o hook seja corrigido ou desativado.

Como Resolver Rapidamente (Escolha uma opção)

Opção 1: Desativar ou remover o plugin de telemetria (Recomendado e mais rápido)
No PowerShell do Windows ou pelo Gerenciador de Arquivos:
1. Feche ou minimize a janela se necessário.
2. Renomeie ou remova temporariamente a pasta do plugin problemático:
   ```powershell
   Rename-Item -Path
"$HOME\.gemini\config\plugins\googlecloudtools.datacloud_telemetry"
-NewName "googlecloudtools.datacloud_telemetry.bak"
   ```
   (Ou desative a extensão no painel de Plugins / Extensões do Antigravity).

Opção 2: Corrigir as aspas no arquivo de hook
1. Abra a pasta:

`C:\Users\Janine\.gemini\config\plugins\googlecloudtools.datacloud_telemetry
\`

2. Abra o arquivo de manifesto / hooks (ex.: `hooks.json`, `plugin.json` ou `gemini-extension.json`).
3. Localize a linha que invoca `telemetry_hook_bundle.js`.
4. Remova as aspas escapadas extras dentro do caminho (mantenha apenas o caminho limpo, ou use caminho relativo simples `./telemetry_hook_bundle.js`).

Plano de Execução do Projeto (Pronto para início imediato)

Assim que você desativar ou corrigir o hook acima e me enviar uma mensagem de confirmação (ex: *"Pronto, corrigido"* ou *"Pode continuar"*), executarei imediatamente todo o escopo solicitado:

```mermaid
flowchart TD
    A[1. Inspeção do Repositório e Workspace] --> B[2. Diagnóstico da Arquitetura]
    B --> C[3. Configuração do venv e Dependências]
    C --> D[4. Execução da Suíte de Testes Inicial]
    D --> E[5. Auditoria e Validação da Porcentagem]
    E --> F[6. Execução e Validação Desktop main.py e Web Vite]
    F --> G[7. Relatório Técnico Final Completo]
```

1. **Inspeção e Arquitetura**:
   - Mapeamento detalhado de `calculator_engine.py`, `expression_parser.py`, `scientific.py`, `finance.py`, `statistics_tools.py`, `fractions_math.py`, `algebra.py`, `geometry.py`, `converters.py`, `matrices.py`, `history.py` e `interface.py`.
   - Análise de segurança (garantindo ausência de `eval` / `exec` inseguros).

2. **Ambiente e Testes**:
   - Validação do Python 3.10+.
   - Ativação/configuração do ambiente virtual `.venv` e instalação rigorosa apenas do `requirements.txt`.
   - Execução integral de `python -m unittest discover tests -v`.

3. **Funcionalidade de Porcentagem**:
   - Validação de cálculos padrão:
     - `10% de 200 = 20`
     - `25% de 100 = 25`
     - `50% de 80 = 40`
     - Aumento: `200 + 10% = 220`

- Desconto: `200 - 10% = 180`
- Correção ou cobertura de testes sem criar duplicação de lógica.

4. **Execução das Interfaces:**
- Validação da inicialização desktop (`python main.py`).
- Verificação da interface web configurada com `package.json` e Vite.

5. **Relatório Técnico Final:**
- Todos os 10 tópicos exigidos consolidados e comprovados por evidências de testes.

AUTORA : Janine Cunha